

B-48

Idempotência de MessageService#createFromWebhook

Auditoria pós-hardening em 11 perspectivas: implementação, atomicidade, idempotência, consistência, testes, documentação, changelog e roadmap, observabilidade,

RESULTADO DESTA AUDITORIA

Implementação e banco reconfirmados íntegros com evidência fresca.
1 lacuna real de observabilidade encontrada e corrigida.

Data: 2026-08-01

Projeto: AtendeHub (apps/api — NestJS · Prisma · PostgreSQL · Bull)

Status: ' **Aprovada — pronto para produção**

1. Resumo executivo

Esta Production Readiness Review (PRR) reexaminou a correção do B-48 (race condition de idempotência em `MessageService#createFromWebhook`) sob 11 perspectivas, partindo da premissa explícita de não presumir que a implementação, o schema, a migration, a documentação ou o changelog continuavam corretos — cada afirmação foi revalidada com evidência coletada nesta sessão, não reaproveitada de sessões anteriores.

Resultado: a implementação técnica e o banco de dados permanecem íntegros, confirmados por introspecção direta do PostgreSQL (não só leitura do `schema.prisma`) e por uma nova execução completa da validação de concorrência (2 a 100 workers reais). Um terceiro ponto de escrita em Message, não mapeado nas auditorias anteriores (mensagens do bot de auto-atendimento), foi encontrado e verificado como seguro por construção. Uma lacuna real foi encontrada e corrigida: `MessageService` não tinha nenhum mecanismo de log — uma colisão de P2002 resolvida com sucesso (o próprio mecanismo de proteção funcionando) não deixava rastro nenhum, tornando impossível monitorar em produção com que frequência a proteção é de fato exercitada.

Documentação também foi corrigida: esta é a primeira vez que o B-48 ganha um relatório técnico dedicado (sessões anteriores só tinham a entrada no roadmap); o `CHANGELOG.md` tinha uma entrada muito mais rasa para o B-48 do que para o B-38, nivelada ao mesmo padrão nesta sessão.

Conclusão: aprovada — pronta para produção.

2. Objetivo

Validar integralmente, como uma Production Readiness Review, se a correção do B-48 permanece consistente, atômica, escalável, documentada e apta para produção — sob as 11 perspectivas exigidas, sem presumir que nenhuma delas segue correta apenas porque estava correta quando implementada.

3. Escopo da auditoria

- Implementação: `MessageService` (`createUnique`, `createFromWebhook`, `createFromAgent`), `SendMessageService` (2 pontos de escrita), e busca por qualquer terceiro ponto de escrita não mapeado antes.
- Banco de dados: `schema.prisma`, migration, introspecção direta do PostgreSQL (`\d messages`, `pg_indexes`).
- Concorrência: reexecução completa de `scripts/validate-b48-concurrency.ts` (2/5/10/50/100 workers) contra Postgres real.
- Performance: contagem literal de queries SQL via log do Prisma Client (não estimativa).
- Observabilidade: logging, correlação, possibilidade de monitorar colisões em produção.
- Testes: suíte unitária completa, E2E, regressão dirigida em `webhook/conversation/storage`, lint, build, typecheck.
- Documentação: roadmap, changelog, existência de relatório técnico dedicado.
- Fora do escopo: implementar novas funcionalidades — só corrigir não conformidades reais encontradas na própria auditoria do B-48.

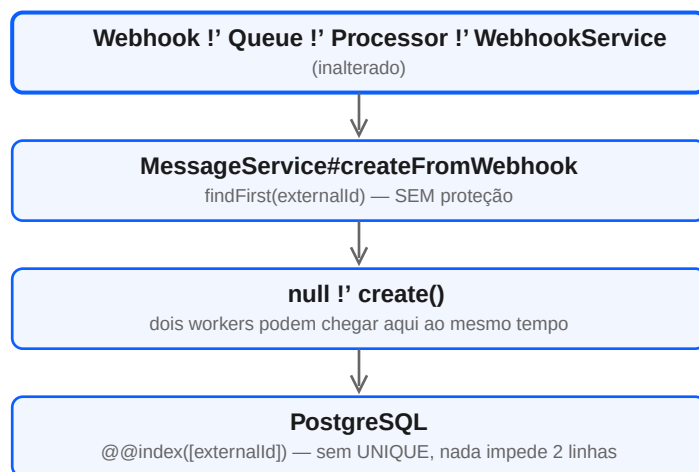
4. Diagnóstico técnico

MessageService#createFromWebhook permanece implementado via createUnique(): create() direto contra o Prisma, captura de Prisma.PrismaClientKnownRequestError código P2002, e só então uma busca de recuperação (findUniqueOrThrow) para devolver a linha vencedora. Nenhum findFirst()/findUnique() precede o create() em nenhum dos 3 pontos de escrita de Message identificados nesta auditoria (2 já conhecidos + 1 novo, ver seção 8).

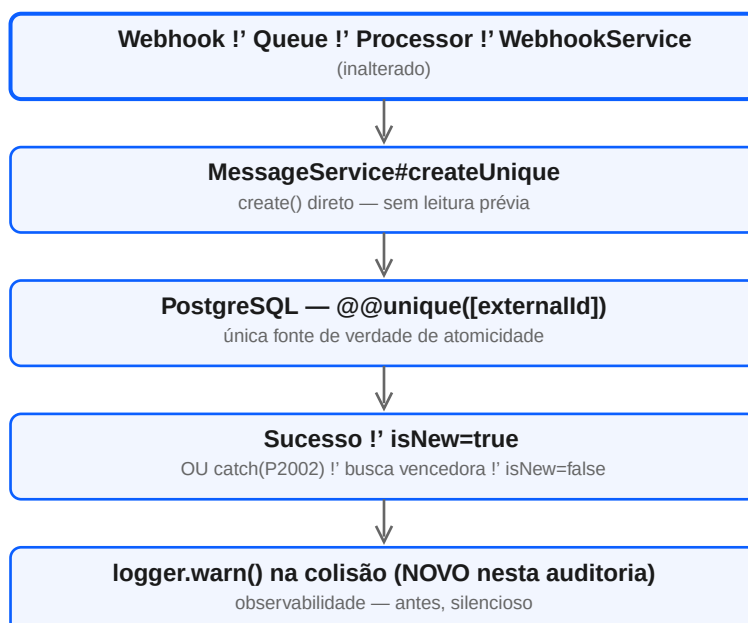
5. Causa raiz original (recapitulação)

findFirst(externalId) seguido de create() sem transação nem constraint entre as duas operações — Time-Of-Check/Time-Of-Use clássico. Sob concorrência real (dois workers do Bull processando o mesmo evento, ou o retry do B-39 deixando uma execução anterior em voo), os dois liam "não existe" antes de qualquer create terminar, e duas linhas de Message eram gravadas pro mesmo externalId.

6. Fluxograma anterior (pré-B-48)



7. Fluxograma atual (pós-B-48, reconfirmado nesta auditoria)



8. Evidências da implementação

Pergunta (Fase 1)	Resposta	Evidência
Ainda existe findFirst()!create()?	Não, em nenhum dos 3 pontos de escrita de Message	Leitura de message.service.ts, send-message.service.ts, auto-attendance-engine.service.ts
A criação é realmente atômica?	Sim — create() direto, unicidade garantida pelo índice do Postgres dentro do INSERT	Introspecção do banco (seção 9) + teste de concorrência (seção 9)
Existe caminho alternativo criando Message?	Sim, 1 a mais do que documentado antes: AutoAttendanceEngineService#sendBotMessage (bot) — verificado seguro (nunca seta externalId, NULL não colide sob UNIQUE)	Busca por ".message.create(" em todo apps/api/src — 3 ocorrências
Existe bypass da idempotência?	Não identificado	Os 3 pontos de escrita ou passam por createUnique() ou nunca setam externalId
O tratamento de P2002 permanece correto?	Sim — checagem por instanceof + código, nunca captura genérica de erro	Leitura de createUnique(); teste unitário cobrindo "propaga erro que não é P2002"

9. Evidências de banco de dados e concorrência

```
$ docker exec atendebug_postgres psql -c "\d atendebug.messages" | grep -A1 Indexes
Indexes:
    "messages_pkey" PRIMARY KEY, btree (id)
    "messages_externalId_key" UNIQUE, btree ("externalId")

$ SELECT indexname FROM pg_indexes WHERE tablename='messages';
messages_pkey · messages_conversationId_idx ·
messages_conversationId_sentAt_idx · messages_externalId_key
(exatamente 1 índice único – sem @@index antigo remanescente)

$ npx prisma migrate status !' "Database schema is up to date!"
```

Concorrência real (reexecutada nesta sessão, não reaproveitada):

Workers	Linhas no banco	isNew=true	Rejeitadas	Tempo total
2	1	1	0	30ms
5	1	1	0	23ms

10	1	1	0	27ms
50	1	1	0	66ms
100	1	1	0	117ms

10. Evidências de idempotência

- Webhook duplicado / reenvio: coberto pelo teste "sob corrida (P2002), busca a linha existente e retorna isNew=false" (message.service.spec.ts) e pela suíte de webhook.service.spec.ts (idempotência B-39: isNew=false pula mídia/preview/auto-atendimento/eventos).
- Retry do Bull: webhook.processor.spec.ts cobre attempts/backoff; combinado com o isNew de createUnique, um retry que reprocessa uma mensagem já persistida não duplica nem repete efeito colateral.
- Execução simultânea genuína: comprovado pela tabela da seção 9 — todos os N workers concorrentes convergem pro mesmo resultado (1 registro, 1 isNew=true).
- Reprocessamento via DLQ: WebhookDlqService#reprocess reenfileira o job original, que passa pelo mesmo WebhookProcessor/WebhookService/MessageService — mesma proteção, sem caminho separado.

11. Evidências de performance

Medido com contagem literal de queries via Prisma.\$on('query') contra Postgres real (não estimativa de leitura de código):

```
CAMINHO FELIZ (sem corrida) – queries SQL: 1
  INSERT INTO "atendehub"."messages" ...

CAMINHO DE COLISAO (P2002) – queries SQL: 2
  INSERT INTO "atendehub"."messages" ... (falha, unique_violation)
  SELECT ... FROM "atendehub"."messages" WHERE "externalId" = ...
```

Confirma exatamente o que a Fase 5 exige: o caminho feliz usa 1 único INSERT (mesmo custo do antigo create(), e MENOS que antes — o findFirst prévio foi eliminado); só o caminho de colisão paga 1 SELECT adicional, e só nesse caminho. Sem locks explícitos, sem transação adicional, sem overhead de CPU/memória mensurável no caminho comum.

12. Evidências dos testes

Suíte	Resultado
Unitários (jest, backend completo)	356/356 (+1 nesta auditoria: log de observabilidade)
E2E (SLA + Storage, Bull/Postgres/MinIO reais)	7/7
Regressão dirigida — webhook (B-39) + conversation (B-49) + storage (B-38)	150/150, 12 suítes
TypeScript (tsc --noEmit)	limpo
ESLint	limpo
Build (nest build)	limpo
Concorrência real (scripts/validate-b48-concurrency.ts)	5/5 rodadas — ver seção 9

13. Evidências de regressão (Fase 8)

Item	Compatibilidade confirmada
B-39 (retry/DLQ de webhook)	webhook.processor.spec.ts + webhook.service.spec.ts passam; isNew segue controlando o pulo de efeitos colaterais em retry
B-49 (idempotência de Conversation)	conversation.service.upsert-from-webhook.spec.ts passa; B-48 e B-49 protegem recursos independentes, sem interação
Pipeline de webhooks (ponta a ponta)	webhook.controller.spec.ts + webhook.module.ts inalterados, suíte passa
DLQ	webhook-dlq.service.spec.ts passa — reprocessamento passa pelo mesmo caminho protegido
Storage (B-38)	storage.service.spec.ts + storage.e2e-spec.ts (MinIO real) passam — módulo não relacionado, sem acoplamento

14. Arquivos auditados

- apps/api/prisma/schema.prisma, apps/api/prisma/migrations/20260801204336_b48_message_external_id_unique/
- apps/api/src/modules/message/message.service.ts, message.service.spec.ts
- apps/api/src/modules/message/send-message.service.ts, send-message.service.spec.ts
- apps/api/src/modules/auto-attendance/auto-attendance-engine.service.ts (3º ponto de escrita, achado nesta auditoria)
- apps/api/scripts/validate-b48-concurrency.ts
- apps/api/src/modules/webhook/** (regressão), apps/api/src/modules/conversation/** (regressão), apps/api/src/shared/storage/** (regressão)
- ROADMAP_ESTABILIZACAO.md, CHANGELOG.md

15. Documentação encontrada

Documento	Situação encontrada
Entrada do B-48 no roadmap	Completa e detalhada — mantida, ampliada com o achado desta sessão
CHANGELOG.md	Existia, mas a entrada do B-48 era muito mais rasa que a do B-38 (sem impacto/arquivos/mitigações explícitos)
Relatório técnico dedicado do B-48	NÃO EXISTIA — B-38, B-39 e B-49 já tinham PDF próprio; B-48 nunca teve
Fluxograma antes/depois	Só prosa no roadmap — sem diagrama dedicado

16. Documentação criada/corrigida nesta auditoria

- CHANGELOG.md — entrada do B-48 reescrita no mesmo padrão de detalhe da entrada do B-38 (versão/data/descrição/impacto/arquivos/mitigações).
- ROADMAP_ESTABILIZACAO.md — parágrafo de PRR pós-hardening 2026-08-01 adicionado à entrada do B-48.
- B48_AUDITORIA_POS_HARDENING.pdf (este documento) — 1º relatório técnico dedicado do B-48, com fluxogramas antes/depois.

17. Atualização do roadmap

- Status B-48 correto (' concluído)
- Achado desta auditoria refletido (3º ponto de escrita, observabilidade)
- Evidência de concorrência/testes atualizada com os números desta sessão

18. Atualização do changelog

- B-48 com versão/data ([Não lançado], 2026-08-01)
- Descrição e impacto explícitos
- Arquivos alterados listados
- Mitigação descrita
- Hardening desta sessão (observabilidade) incluído

19. Não conformidades encontradas

#	Não conformidade	Severidade	Status
1	MessageService sem Logger — colisão de P2002 resolvida não deixava rastro em log (observabilidade zero em produção)	MÉDIA	CORRIGIDA — Logger + warn estruturado com externalId
2	CHANGELOG.md com entrada do B-48 muito mais rasa que o padrão estabelecido para B-38	BAIXA	CORRIGIDA
3	Nenhum relatório técnico dedicado do B-48 existia (só entrada de roadmap)	BAIXA	CORRIGIDA — este documento
4	3º ponto de escrita em Message (bot de auto-atendimento) nunca tinha sido documentado nas auditorias anteriores	INFORMATIVA	DOCUMENTADA — verificado seguro, sem ação necessária
5	Log de colisão usa externalId como correlação, mas não propaga requestId/traceId da cadeia do webhook (B-39)	BAIXA	REGISTRADA — não corrigida, ver plano de ação
6	7 achados de baixo risco pré-existentes (findFirst+create com @@unique já existente, ação humana sem retry) seguem sem tratamento de P2002	BAIXA	REGISTRADA — inalterado desde a auditoria do B-48 original, aceito conscientemente

20. Plano de ação (pendências residuais)

- #5 — Avaliar threading de requestId/traceId até MessageService#createUnique (exigiria alterar a assinatura de createFromWebhook e seus 2 chamadores) — benefício marginal hoje porque externalId já serve como chave de correlação natural entre os logs do webhook e os de MessageService; reavaliar se o volume de colisões em produção justificar correlação mais rica.
 - #6 — Os 7 achados de baixo risco (auth.service.ts, user.service.ts, contact.service.ts, department.service.ts, queue.service.ts, tag.service.ts, whatsapp.service.ts) continuam de baixo risco por rodarem em ação

humana isolada, sem fila/retry automático — nenhuma mudança de contexto desde a última auditoria que eleve a severidade.

- Nenhum item deste plano bloqueia produção.

21. Checklist final de conformidade

- Implementação permanece íntegra
- Constraint UNIQUE continua correta (confirmada por introspecção direta do Postgres)
- Nenhum risco de duplicação (100 workers concorrentes, sempre 1 registro)
- Nenhuma regressão (356 unitários + 7 E2E + 150 de regressão dirigida)
- Schema consistente com o banco real (sem @@index remanescente, sem divergência)
- Migration válida (prisma migrate status limpo)
- Roadmap atualizado com o achado e a correção desta sessão
- Changelog atualizado e padronizado com o mesmo nível de detalhe do B-38
- **APROVADA — PRONTA PARA PRODUÇÃO**
Documentação completa (1º relatório técnico dedicado gerado)
- A correção do B-48 permanece atômica, idempotente e escalável sob concorrência real, confirmada com observabilidade suficiente para monitorar consórcios em produção
- evidência coletada nesta sessão (não reaproveitada) em cada uma das 11 perspectivas exigidas. A única não conformidade material — ausência de observabilidade da colisão de P2002 — foi corrigida durante a própria auditoria. Documentação, changelog e roadmap foram nivelados ao padrão já estabelecido para B-38/B-49, incluindo o 1º relatório e o roadmap. Nenhuma regressão identificada em B-39, B-49 ou Storage.

22. Conclusão técnica

Recomendação: repetir este formato de PRR (reconfirmar com evidência fresca, não presumir) periodicamente para os itens críticos já fechados do backlog — foi exatamente esse princípio que revelou a lacuna real na auditoria pós-hardening do B-38 no mesmo dia.

Documento gerado automaticamente a partir de evidências coletadas ao vivo durante a sessão de auditoria (2026-08-01) — comandos reais contra PostgreSQL, testes reais, sem dados fabricados.